

Capstone 4 — Domain B: B2B Order Lifecycle Pipeline

Build a 4-agent pipeline that manages the full B2B order lifecycle: intake validation, fulfillment planning with HITL for high-value splits, exception monitoring with carrier API circuit breaker, and proactive customer communication.

Project Brief

BUSINESS CONTEXT

A B2B distributor processes 200+ purchase orders daily. In Capstone 3, you built a ReAct agent that investigates individual exceptions. But the full order lifecycle — from PO arrival through delivery confirmation — involves four distinct operational stages, each requiring different skills and data sources. A single agent handling everything creates a monolith that can't scale, can't be independently tested, and can't be safely modified without risk to the entire pipeline.

The production solution: **four specialized agents** in a pipeline. The Order Intake Agent validates and enriches POs. The Fulfillment Planning Agent allocates inventory and plans shipments. The Exception Monitor Agent watches for delays and issues. The Customer Communication Agent sends proactive updates.

Two production patterns make this pipeline trustworthy: **HITL** where the operations manager approves split shipments over \$50K or exception resolutions involving credit issuance, and a **circuit breaker** that trips when the carrier tracking API fails 5+ times in 10 minutes, switching to fallback polling mode.

WHAT YOU WILL BUILD

- **Agent 1 — Order Intake:** Validates PO format, checks customer credit, enriches line items with contract pricing. Input guardrails: duplicate PO detection, schema validation.
- **Agent 2 — Fulfillment Planning:** Checks multi-warehouse inventory, creates shipment plans. If split shipment >\$50K, pauses for operations manager HITL approval.
- **Agent 3 — Exception Monitor:** Polls carrier tracking APIs, detects delays/exceptions. Circuit breaker: 5 carrier failures in 10 min → fallback polling mode. If resolution involves credit/refund issuance, requires ops manager HITL approval.
- **Agent 4 — Customer Communication:** Drafts proactive updates (confirmation, shipment, delay, delivery). Output guardrails: SLA compliance check, tone matching per customer tier.

Skills practiced: Multi-agent architecture (M14), HITL (M17), guardrails (M16–M17), circuit breaker, evaluation (M18), structured inter-agent state passing.

Prerequisites

Complete these modules before starting this capstone:

- **M03–M05 — Tool Use Fundamentals:** You need tool definitions, structured outputs, and basic tool orchestration. Every agent in this pipeline uses tool calls.
- **M09 — RAG (Retrieval-Augmented Generation):** Agent 2 retrieves product catalog and contract data to verify pricing. Understanding RAG patterns helps you extend the pipeline later.
- **M12 — ReAct Pattern:** Each agent internally uses an observe → think → act loop. The Exception Monitor agent is essentially a ReAct agent that reasons about anomalies.
- **M14 — Multi-Agent Systems:** The 4-agent pipeline architecture, structured state passing between agents, and agent handoff patterns come directly from M14.
- **M16 — Input Guardrails:** Agent 1 uses input validation (PO schema checks, duplicate detection) and Agent 4 applies output guardrails (SLA compliance, tone matching). M16 teaches these patterns.
- **M17 — Human-in-the-Loop & Guardrails:** The HITL approval flow for split shipments and credit resolutions, plus the circuit breaker pattern, are covered in M17.

- **M18 — Evaluation:** The pytest-based test suite at the end of the capstone uses M18's scoring + harness patterns to validate end-to-end accuracy.

You should also be comfortable with Python classes, JSON data structures, and running scripts from the command line.

Environment Setup

WHAT YOU'LL BUILD

A multi-agent B2B order lifecycle pipeline with four specialized agents, human-in-the-loop escalation for high-value splits and credit resolutions, and a time-windowed circuit breaker for carrier API failures.

Time estimate: 2–3 hours Difficulty: ★★★★★☆

Requirements

- Python 3.10 or higher
- An Anthropic API key (get one at console.anthropic.com)

Install Everything

Copy and paste this entire block into your terminal:

UNIX / MACOS · **WINDOWS**

UNIX / MACOS

```
mkdir order-lifecycle-pipeline && cd order-lifecycle-pipeline
python3 -m venv venv && source venv/bin/activate
pip install "anthropic≥0.30.0" pytest
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
mkdir order-lifecycle-pipeline && cd order-lifecycle-pipeline
python -m venv venv && venv\Scripts\activate
pip install "anthropic≥0.30.0" pytest
set ANTHROPIC_API_KEY=your-key-here
```

Checkpoint

Run `python -c "import anthropic; print('OK')"`. You should see `OK`. If you see `ModuleNotFoundError`, make sure your virtual environment is activated.

File Structure

Here is every file you will create in this capstone. The solution consolidates all agents, state, and circuit breaker logic into `pipeline.py` for simplicity — no separate `agents/` directory needed.

```
order-lifecycle-pipeline/
├─ mock_tools.py      # All mock tool implementations (ERP, inventory, carrier, messaging)
├─ pipeline.py        # All 4 agents + state + circuit breaker + orchestrator
└─ test_pipeline.py   # Test suite (5 scenarios, pytest)
```

Domain Glossary

Split Shipment

When an order is fulfilled from multiple warehouses due to inventory distribution. Creates multiple tracking numbers. Shipments over \$50K require operations manager approval.

Circuit Breaker

Safety mechanism that trips after 5 carrier API failures in 10 minutes. Switches to fallback polling mode (check every 30 min instead of real-time). Prevents cascade failures.

EDI 850

Electronic Data Interchange format for Purchase Orders. Standardized B2B document format. The Intake Agent must parse and validate EDI-formatted POs alongside email and portal submissions.

Credit Check

Verification that the customer's account has sufficient credit for the order. Available credit = credit limit - outstanding balance. Failed credit checks block order processing.

Backorder

When a line item's requested quantity exceeds available inventory. The item is placed on backorder with an estimated restock date. May trigger a split shipment.

Customer Tier

Classification (Gold/Silver/Bronze) determining communication priority, SLA commitments, and escalation thresholds. Gold customers get proactive updates; Bronze gets standard notifications.

Fallback Polling

When the circuit breaker trips, the system switches from real-time carrier API calls to periodic batch polling (every 30 minutes). Degraded but functional monitoring.

Pipeline State

A typed object flowing through all 4 agents. Each agent reads predecessor outputs and writes its own. The single source of truth for order processing status.

Architecture

4-AGENT PIPELINE — B2B ORDER LIFECYCLE

HITL — SPLIT SHIPMENT APPROVAL (>\$50K)

CIRCUIT BREAKER — CARRIER API (5 FAILURES / 10 MIN)



Mock Data Specification

PIPELINE STATE

PIPELINE STATE

```
{
  "po_number": "P0-2024-11234",
  "stage": "intake",
  "created_at": "2024-03-10T08:00:00Z",

  "intake_output": {
    "validated": true,
    "customer_verified": true,
    "credit_check_passed": true,
    "duplicate_detected": false,
    "line_items_enriched": [
      {"sku": "SKU-4892", "qty": 50, "unit_price": 495.00,
       "contract_price_verified": true, "available": true},
      {"sku": "SKU-7721", "qty": 200, "unit_price": 125.00,
       "contract_price_verified": true, "available": false}
    ],
    "total_value": 49750.00,
    "priority": "standard"
  },

  "fulfillment_output": {
    "plan_id": "FP-2024-11234",
    "requires_split": true,
    "shipments": [
      {"warehouse": "East", "skus": ["SKU-4892"], "value": 24750.00,
       "estimated_ship_date": "2024-03-12", "carrier": "UPS"},
      {"warehouse": "West", "skus": ["SKU-7721"], "value": 25000.00,
       "estimated_ship_date": "2024-03-15", "carrier": "FedEx"}
    ],
    "total_value": 49750.00,
    "human_approval_required": false
  },

  "exception_output": null,
  "communication_output": null,

  "circuit_breaker": {
    "carrier_api_failures": 0,
    "window_start": null,
    "threshold": 5,
    "window_minutes": 10,
    "status": "healthy"
  }
}
```

Step-by-Step Build Guide

Now you understand the architecture and data flow. Let's build it piece by piece. Each step creates one file, gives you a command to test it, and shows the expected output. Follow the steps in order — later steps depend on earlier ones.

Step 1: Create the Mock Tools (mock_tools.py)

What & Why: All four agents need mock tool functions (ERP queries, inventory checks, carrier tracking, messaging). Building them first means you can test each agent independently. The full mock tools code is shown in the Mock Tool Implementations section below.

Create the file `mock_tools.py` using the code in the Mock Tool Implementations section.

Run:

```
python -c "
from mock_tools import validate_purchase_order, verify_customer_credit
```

```
po = {'po_number': 'PO-TEST', 'customer': 'Acme', 'line_items': []}
print(validate_purchase_order(po))
print(verify_customer_credit('Acme Manufacturing', 50000))
"
```

Expected output:

```
{'validated': True, 'errors': [], 'normalized_po': {'po_number': 'PO-TEST', 'customer': 'Acme',
'line_items': []}}
{'credit_approved': True, 'credit_limit': 100000, 'available_credit': 65000, 'payment_terms': 'Net
45'}
```

Checkpoint

If both outputs show `validated: True` and `credit_approved: True`, Step 1 is working. If you see `CUSTOMER_NOT_FOUND`, make sure you're passing the exact customer name `"Acme Manufacturing"`.

Step 2: Build the Complete Pipeline (pipeline.py)

What & Why: The pipeline file contains everything: the `OrderPipelineState` dataclass, the time-windowed `CarrierCircuitBreaker`, all four agent functions (Intake with Claude reasoning, Fulfillment, Exception Monitor, Communication with Claude drafting), HITL approval flows, and the orchestrator that wires them together. Consolidating into one file keeps the project simple — no import headaches, no package structure to manage.

Create `pipeline.py` using the complete solution code in the Complete Solution section below.

Run:

```
python pipeline.py
```

Then type `demo` at the prompt.

Expected output:

```
[INTAKE] Processing PO-2024-11234...
[INTAKE] Claude validated order and extracted key fields.
[INTAKE] Complete.
[FULFILLMENT] Planning shipments...
[FULFILLMENT] Complete. Split: True.
[MONITOR] Checking shipment status...
[MONITOR] Complete.
[COMMUNICATION] Drafting updates...
[COMMUNICATION] Claude drafted customer notification.
[PIPELINE] Complete! Order PO-2024-11234 processed.
```

Checkpoint

If you see `[PIPELINE] Complete!`, the orchestrator is working end-to-end. If the pipeline stops at `[INTAKE] FAILED`, check that the sample PO uses customer name `"Acme Manufacturing"` (must match the credit database). If you see `AuthenticationError`, verify your `ANTHROPIC_API_KEY` is set correctly.

Step 3: Test HITL for Split Shipments & Credit Resolutions

What & Why: When the fulfillment plan requires a split shipment exceeding \$50K, the pipeline pauses. Present the operations manager with three options: approve split, wait for consolidation, or partial ship with backorder cancellation. Additionally, when

the Exception Monitor's resolution involves issuing a credit or refund, the pipeline pauses for ops manager approval before proceeding.

The HITL functions (`split_shipment_review` and `credit_resolution_review`) are already included in `pipeline.py` . Test by modifying the sample PO to have a total value over \$50K:

Run (to trigger HITL):

```
python -c "  
from pipeline import run_order_pipeline  
po = {'po_number': 'PO-HITL-TEST', 'customer': 'Acme Manufacturing',  
      'line_items': [  
        {'sku': 'SKU-4892', 'qty': 60, 'unit_price': 495.00},  
        {'sku': 'SKU-7721', 'qty': 200, 'unit_price': 125.00}]}  
run_order_pipeline(po)  
"
```

Expected behavior: If the total value exceeds \$50K and a split shipment is required, the pipeline pauses and prompts for operations manager approval. Enter `1` to approve the split.

Checkpoint

If you see `OPERATIONS MANAGER REVIEW REQUIRED` , HITL is working. If the pipeline runs straight through without pausing, check the `human_approval_required` logic: it should be `requires_split AND total > 50000` .

Step 4: Create the Test Suite (test_pipeline.py)

What & Why: The test suite runs 5 scenarios covering happy path, edge cases, and adversarial inputs. The complete test code is in the Testing Guide section below.

Run:

```
pytest test_pipeline.py -v
```

Expected output:

```
test_pipeline.py::test_happy_path_standard_order PASSED  
test_pipeline.py::test_credit_check_failure PASSED  
test_pipeline.py::test_hitl_escalation_high_value PASSED  
test_pipeline.py::test_circuit_breaker_trip PASSED  
test_pipeline.py::test_invalid_input_missing_fields PASSED  
  
===== 5 passed =====
```

Checkpoint

If all 5 tests pass, your pipeline is production-ready. If a test fails, check the Testing Guide table for expected behavior and compare against your implementation.

Mock Tool Implementations

PYTHON (ALL TOOLS) · NODE.JS/TYPESCRIPT (ALL TOOLS)

PYTHON (ALL TOOLS)

```
FROM datetime import datetime, timedelta

# == INTAKE AGENT ==

CUSTOMER_CREDIT = {
    "Acme Manufacturing": {"limit": 100000, "outstanding": 35000, "terms": "Net 45"},
    "Beta Industrial": {"limit": 50000, "outstanding": 48000, "terms": "Net 30"},
}
PROCESSED_POS = set()

FUNCTION validate_purchase_order(raw_po):
    required = ["po_number", "customer", "line_items"]
    missing = [f FOR f IN required IF f not in raw_po]
    IF missing:

        # ... (full source in the desktop HTML) ...

        "urgency": "high" if update_type == "delay" else "standard"}

FUNCTION send_customer_update(draft_id, channel, customer_contact):
    RETURN {"message_id": f"MSG-{draft_id}",
            "sent_at": datetime.now().isoformat(),
            "delivery_status": "delivered"}
```

NODE.JS/TYPESCRIPT (ALL TOOLS)

```
// mock_tools.ts - Tools for all 4 pipeline agents.

// == INTAKE AGENT ==

CONST CUSTOMER_CREDIT: Record<string, any> = {
    "Acme Manufacturing": { limit, outstanding, terms: "Net 45" },
    "Beta Industrial": { limit, outstanding, terms: "Net 30" },
};
CONST PROCESSED_POS = new Set<string>();

export FUNCTION validatePurchaseOrder(rawPo): any {
    CONST required = ["po_number", "customer", "line_items"];
    CONST missing = required.filter(f => !rawPo[f]);
    IF (missing.length) return { validated, errors: missing };

    # ... (full source in the desktop HTML) ...

    RETURN {
        message_id: `MSG-${draftId}`,
        sent_at: NEW Date().toISOString(),
        delivery_status,
    };
}
```

Complete Solution

The orchestrator runs agents sequentially, checking the circuit breaker and HITL conditions at each transition. The time-windowed circuit breaker is the key architectural difference from Capstone 3-B's simple counter.


```
import json, anthropic
from datetime import datetime, timedelta
from dataclasses import dataclass, field, asdict

# — Time-Windowed Circuit Breaker —————

class CarrierCircuitBreaker:

    failures= field(default_factory=list) # timestamps
    threshold= 5
    window_minutes= 10
    status= "healthy" # healthy | tripped | fallback

    def record_failure(self):

        # ... (full source in the desktop HTML) ...

        if cmd.lower() == "demo":
            state = run_order_pipeline(sample)
            print(f"\nFinal state:\n{json.dumps(asdict(state), indent=2, default=str)}")

if __name__ == "__main__":
    main()
```

```
// pipeline.ts — B2B Order Lifecycle Pipeline (Capstone 4-B)
// Usage=... npx ts-node pipeline.ts

import Anthropic from "@anthropic-ai/sdk";
import * as readline from "readline";
import {
    validatePurchaseOrder,
    verifyCustomerCredit,
    enrichLineItems,
    createShipmentPlan,
    pollShipmentStatus,
    detectExceptions,
    draftOrderUpdate,
    sendCustomerUpdate,

    # ... (full source in the desktop HTML) ...

    console.log(`\nFinal state:\n${JSON.stringify(state, null, 2)}`);
}
}
}

main().catch(console.error);
```

What Just Happened?

You built a 4-agent pipeline with Claude API integration and production safety patterns: (1) **Agent 1 (Intake)** uses **Claude with tool use** to validate orders and check credit through an agentic loop; (2) **Agent 4 (Communication)** uses **Claude with tool use** to draft and send professional customer notifications; (3) **Agents 2 and 3 are deterministic** — inventory allocation and status polling don't need LLM reasoning; (4) **HITL for split shipments** — the pipeline pauses when a split exceeds \$50K; (5) **Time-windowed circuit breaker** — only counts failures within a 10-minute sliding window, preventing false trips from infrequent transient errors.

Expected Output — Standard Flow

```
[INTAKE] Processing PO-2024-11234...
[INTAKE] Claude validated order and extracted key fields.
[INTAKE] Complete.
```

```
[FULFILLMENT] Planning shipments...
[FULFILLMENT] Complete. Split: True.

[MONITOR] Checking shipment status...
[MONITOR] Complete.

[COMMUNICATION] Drafting updates...
[COMMUNICATION] Claude drafted customer notification.
[PIPELINE] Complete! Order PO-2024-11234 processed.

Final state:
  po_number: PO-2024-11234
  stage: complete
  fulfillment_output.requires_split: true
  circuit_breaker.status: healthy
```

Testing Guide

test_pipeline.py — Complete Test Suite

Create `test_pipeline.py` with these 5 pytest test cases. They use `unittest.mock.patch` to stub Claude API calls so tests run without an API key:

PYTHON (PYTEST)

PYTHON (PYTEST)

```
IMPORT pytest
FROM unittest.mock import patch, MagicMock
FROM pipeline import (
    run_order_pipeline, CarrierCircuitBreaker, OrderPipelineState
)
FROM mock_tools import PROCESSED_POS

FUNCTION _stub_claude(*args, **kwargs):

    mock_resp = MagicMock()
    mock_resp.stop_reason = "end_turn"
    text_block = MagicMock()
    text_block.type = "text"
    text_block.text = "Order validated successfully."

    # ... (full source in the desktop HTML) ...

    # Missing 'customer' and 'line_items' fields
    po = {"po_number": "PO-TEST-005"}
    state = run_order_pipeline(po)

    assert state.stage == "error", "Pipeline should reject orders with missing fields"
```

Run:

```
pytest test_pipeline.py -v
```

Expected output:

```
test_pipeline.py::test_happy_path_standard_order PASSED
test_pipeline.py::test_credit_check_failure PASSED
test_pipeline.py::test_hitl_escalation_high_value PASSED
test_pipeline.py::test_circuit_breaker_trip PASSED
test_pipeline.py::test_invalid_input_missing_fields PASSED
```

===== 5 passed =====

Test Scenario Matrix

TYPE	SCENARIO	EXPECTED BEHAVIOR
HAPPY	Standard order, single warehouse	All 4 agents run, confirmation email sent, no HITL
HAPPY	Split shipment under \$50K	Auto-approved, two shipment plans created, customer notified
HAPPY	Split shipment over \$50K	Pipeline pauses for ops manager, manager approves, pipeline resumes
HAPPY	Shipment delay detected	Exception monitor catches it, communication agent sends delay notification
HAPPY	Full lifecycle through delivery	All stages logged, delivery confirmation sent
EDGE	Credit check fails (Beta Industrial, \$48K outstanding)	Pipeline stops at intake with rejection
EDGE	Manager selects “wait for consolidation”	Pipeline pauses at fulfillment stage, no shipment created
EDGE	SKU discontinued mid-fulfillment	Exception monitor catches, suggests substitute
ADVERSARIAL	Carrier API fails 6 times in 10 minutes	Circuit breaker trips, switches to fallback polling, ops alerted
ADVERSARIAL	Duplicate PO submitted	Intake agent detects duplicate, prevents double-processing

Verify Everything Works

Run the complete pipeline end-to-end with a single command:

UNIX / MACOS - WINDOWS

UNIX / MACOS

```
python -c "  
FROM pipeline import run_order_pipeline  
FROM dataclasses import asdict  
IMPORT json  
  
sample = {'po_number': 'PO-2024-11234', 'customer': 'Acme Manufacturing',  
          'line_items': [  
              {'sku': 'SKU-4892', 'qty': 50, 'unit_price': 495.00},  
              {'sku': 'SKU-7721', 'qty': 200, 'unit_price': 125.00}]}  
state = run_order_pipeline(sample)  
print(json.dumps(asdict(state), indent=2, default=str))  
" && pytest test_pipeline.py -v
```

WINDOWS

```
python -c "from pipeline import run_order_pipeline; from dataclasses import asdict; import json; sample = {'po_number':  
'PO-2024-11234', 'customer': 'Acme Manufacturing', 'line_items': [{'sku': 'SKU-4892', 'qty': 50, 'unit_price': 495.00},  
{'sku': 'SKU-7721', 'qty': 200, 'unit_price': 125.00}]}; state = run_order_pipeline(sample);  
print(json.dumps(asdict(state), indent=2, default=str))" && pytest test_pipeline.py -v
```

Expected final output: The pipeline runs all 4 agents (with Claude API calls for Intake and Communication) on the sample PO, processes the split shipment, and completes. The pytest suite then runs all 5 tests (mocking Claude calls) and reports 5 passed.

```
[INTAKE] Processing PO-2024-11234...  
[INTAKE] Claude validated order and extracted key fields.  
[INTAKE] Complete.  
[FULFILLMENT] Planning shipments...  
[FULFILLMENT] Complete. Split: True.  
[MONITOR] Checking shipment status...  
[MONITOR] Complete.  
[COMMUNICATION] Drafting updates...  
[COMMUNICATION] Claude drafted customer notification.  
[PIPELINE] Complete! Order PO-2024-11234 processed.
```

```
Final state:
  po_number: P0-2024-11234
  stage: complete
  fulfillment_output.requires_split: true

test_pipeline.py::test_happy_path_standard_order PASSED
test_pipeline.py::test_credit_check_failure PASSED
test_pipeline.py::test_hitl_escalation_high_value PASSED
test_pipeline.py::test_circuit_breaker_trip PASSED
test_pipeline.py::test_invalid_input_missing_fields PASSED

===== 5 passed =====
```

CONGRATULATIONS!

You have built a complete multi-agent B2B order lifecycle pipeline with four specialized agents — two powered by Claude API calls with tool use (Intake and Communication) and two deterministic (Fulfillment and Exception Monitor). The pipeline includes a time-windowed circuit breaker that switches to fallback polling when the carrier API fails, and human-in-the-loop review for high-value split shipments and credit resolutions. This is the same architecture used in production order management systems — the only differences in a real deployment would be connecting to actual ERP, WMS, and carrier APIs instead of mock data.

Compliance Notes

⚠️ PCI-DSS, EDI & CONTRACT CONFIDENTIALITY

A multi-agent order pipeline handles sensitive commercial data at every stage: customer credit limits, contract pricing tiers, inventory levels, and supplier quality information.

- **Credit data protection:** Customer credit limits and outstanding balances are highly sensitive. Never expose one customer's credit data to another. Encrypt at rest.
- **EDI audit trail:** If POs arrive via EDI 850, every transformation (intake validation, enrichment) must be logged for EDI audit compliance.
- **HITL decision logging:** Operations manager split shipment and credit/refund resolution decisions are business records. Log decision, rationale, reviewer identity, and timestamp.
- **Carrier API credentials:** Carrier tracking API keys must be stored securely (environment variables or secrets manager), never in code or pipeline state.
- **Circuit breaker alerts:** When the breaker trips, the alert to operations must NOT include order details or customer names — only the failure count and affected service.

💰 COST: 2 CLAUDE API CALLS PER ORDER

Each pipeline run makes 2 Claude API calls (Agent 1: Intake validation, Agent 4: Communication drafting). Agents 2 and 3 are deterministic — no LLM needed for inventory allocation or status polling. Typical consumption: 3,000–6,000 tokens total. At Sonnet pricing, budget ~\$0.02–\$0.04 per order. For 200 orders/day: \$4–\$8/day. The HITL path adds human time but no additional API cost. The circuit breaker in fallback mode *reduces* carrier API cost (fewer polling calls).

Troubleshooting

Common Errors

IMPORT ERRORS

ModuleNotFoundError: No module named 'anthropic' — Your virtual environment is not activated. Run `source venv/bin/activate` (Unix) or `venv\Scripts\activate` (Windows).

ModuleNotFoundError: No module named 'mock_tools' — You are running the script from the wrong directory. Make sure you are inside `order-lifecycle-pipeline/` where both `pipeline.py` and `mock_tools.py` exist.

API ERRORS

AuthenticationError: 401 — Your `ANTHROPIC_API_KEY` is not set or is invalid. Run `echo $ANTHROPIC_API_KEY` (Unix) or `echo %ANTHROPIC_API_KEY%` (Windows) to verify.

RateLimitError: 429 — You are sending too many requests. Wait a few seconds and retry. For testing, the pipeline uses mock tools so API calls are minimal.

PIPELINE LOGIC ERRORS

Credit check always fails — Make sure the customer name in your PO exactly matches a key in `CUSTOMER_CREDIT` (e.g., `"Acme Manufacturing"`, not `"Acme"`). The lookup is case-sensitive.

HITL never triggers — The approval check requires both conditions: `requires_split == True` AND `total_value > 50000`. If your test PO total is under \$50K, HITL will be skipped even on a split shipment.

Circuit breaker doesn't trip — Ensure you are recording failures in rapid succession (within 10 minutes). If you add `time.sleep()` between failures for testing, the window may expire before reaching the threshold.

Duplicate PO not detected — The `PROCESSED_POS` set is module-level. If you restart the Python process between tests, the set is empty. Duplicate detection only works within a single process run.